

Computer Architecture and Organisation

CSC 212

TABLE OF CONTENTS

Table of Contents.....	i
CHAPTER 4: Digital Logic and Digital Systems	1
4.1 Introduction	1
4.2 Logic Operators.....	1
4.3 Logic Gates and Circuits	3
4.3.1 Logic Circuits Design	6
4.3.2 Exercise	7
4.3.3 Truth Tables.....	9
4.3.4 Exercise.....	10
4.3.5 Disjunctive Normal Form	10
4.4 Basic Digital Logic Circuits	12
4.4.1 Combinational Circuits	13
4.4.1.1 The Half Adder	14
4.4.1.2 The Full Adder	15
4.4.1.3 A Circuit to Add Two 4-Bit Binary Numbers	17
4.4.1.4 A circuit that adds one to a 4-bit binary number	18
4.4.1.5 A circuit that finds the ones complement of a 4-bit binary number	18
4.4.2 Sequential Logic Elements.....	18
4.5 Boolean Algebra.....	20
4.5.1 Comments on the rules.....	21
4.5.2 Exercise	23
4.5.3 The NAND Gate	23
4.5.4 Exercise.....	24
Rules for Boolean Algebra.....	25
Logic Gates Symbols.....	26

CHAPTER 4: DIGITAL LOGIC AND DIGITAL SYSTEMS

4.1 INTRODUCTION

- A computer is built up of millions of tiny switches called **transistors**. The tiny transistors work together to produce output. They have the property that they can be wired together in such a way that an output from one switch can turn another switch on or off.
- As a computer computes, these switches turn each other on or off in a pattern determined both by the way they are wired together and by the program that the computer is executing.
- Machine language instructions are expressed as binary numbers. Remember a binary number is made up of just two possible digits, zero and one. So, a machine language instruction is just a sequence of zeros and ones.
- A computer can work directly with binary numbers because switches can readily represent such numbers, for example:
 - One can represent turning the switch on and;
 - Zero can represent turning the switch off.
- It follows that machine language instructions are stored in memory as patterns of switches turned on or off.
- When a machine language instruction is loaded into the CPU, all that happens is that certain switches are turned on or off in the pattern that encodes that particular instruction.
- The CPU is built to respond to this pattern by executing the instruction it encodes; it does this simply because of the way all the other switches in the CPU are wired together.
- In this chapter we are going to look at how these switches are created.

4.2 LOGIC OPERATORS

- George Boole was a mathematician and logician who developed ways of expressing logical processes using algebraic symbols thus creating a branch of mathematics known as Boolean Algebra.
- Boolean algebra is an algebra for the manipulation of objects that can take on only two values, typically true and false. These values are known as Boolean values.
- In Boolean Algebra logical operators operates Boolean value(s) and returns an output of another Boolean value.
- The **basic logic operators** are: **NOT, AND, OR**
- The **NOT** operator takes in only **one input** while all the **other operators** takes in at least **two inputs**. From the input the operators produce a single output.
- A truth table is used to show the outcomes (output) of all possible combinations of inputs that can be operated on by an operator.
- In truth tables, **1 represents TRUE** and **0 represents FALSE**.
- For the NOT operator, if **X = 1 (i.e. X is true)** then **NOT X = 0 (i.e. NOT X is false)** and **vice-versa**. The **truth table** for the **NOT** operator is as follows:

X	NOT X
1	0
0	1

- The **AND** operator takes at least 2 inputs. **X AND Y** is true only if X and Y are both true. In all other cases X AND Y is false:

X	Y	X AND Y
0	0	0
0	1	0
1	0	0
1	1	1

- The **OR** operator takes at least 2 inputs as well. **X OR Y** is false if both X and Y are both false. In all other cases, X OR Y is true.

X	Y	X OR Y
0	0	0
0	1	1
1	0	1
1	1	1

- Additional operators include: (Exclusive OR) **XOR**, **NAND**, **NOR**, (Exclusive NOR) **XNOR**
- The **EXCLUSIVE OR**, written as **XOR** also takes at least 2 inputs. In general, **X XOR Y** is true if X is true or Y is true, but not both.

X	Y	X XOR Y
0	0	0
0	1	1
1	0	1
1	1	0

- The **NAND** gate operates as an **AND** gate followed by a **NOT(inverter)** gate. It acts in the manner of the logical operation “AND” followed by negation. The output is “false” if both inputs are “true”. Otherwise, the output is “true”.

X	Y	X NAND Y
0	0	1
0	1	1
1	0	1
1	1	0

- The **NOR** gate is a combination **OR** gate followed by an inverter. Its output is “true” if both inputs are “false”. Otherwise, the output is “false”.

X	Y	X NOR Y
0	0	1
0	1	0
1	0	0
1	1	0

- The **XNOR** (exclusive-NOR) gate is a combination of **XOR** gate followed by an inverter. Its output is “true” if the inputs are the same, and “false” if the inputs are different.

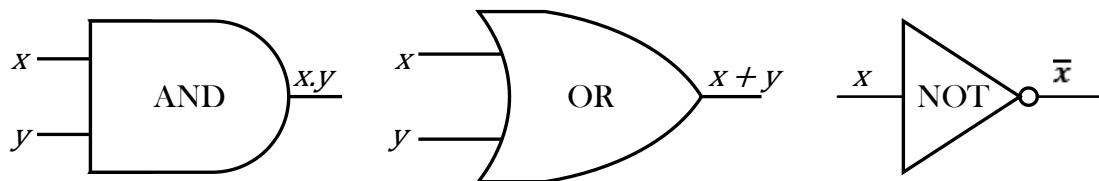
X	Y	X XNOR Y
0	0	1
0	1	0
1	0	0
1	1	1

- The order of precedence of these operators is:
 - Brackets
 - NOT
 - AND
 - OR
 - XOR
- *Meaning that, if there are no brackets, NOT is done before AND, which is done before OR and XOR is done last.*
- For example, we can check whether the following expression is TRUE or FALSE:
 $6 < 5 \text{ OR } \text{NOT } 5 < 6 \text{ AND } 7 < 8 \text{ XOR } 2 < 3 \text{ AND } 6 < 10$
- Rewriting this, with 1 for the true expressions and 0 for the false expressions gives:
 $0 \text{ OR } \text{NOT } 1 \text{ AND } 1 \text{ XOR } 1 \text{ AND } 1$
- Doing the NOT first gives:
 $0 \text{ OR } 0 \text{ AND } 1 \text{ XOR } 1 \text{ AND } 1$
- Next we do the ANDs:
 $0 \text{ OR } 0 \text{ XOR } 1$
- Now doing the OR to get:
 $0 \text{ XOR } 1 = 1$ *so the expression is true.*
- There is a notation used for convenience when dealing with these logical operators:
 - NOT X is written as $\bar{x}$ or x'
 - X AND Y is written as $x.y$ or xy
 - X OR Y is written as $x + y$
 - X NAND Y is written as $\overline{x.y}$ or $(x.y)'$
 - X NOR Y is written as $\overline{x + y}$ or $(x + y)'$
 - X XOR Y is written as $x \oplus y = \bar{x}y + x\bar{y}$ or $x'y + xy'$
 - X XNOR Y is written as $\overline{x \oplus y} = \bar{\bar{x}y + x\bar{y}} = \bar{\bar{x}y} + \bar{x\bar{y}} = \bar{\bar{x}}\bar{y} + x\bar{\bar{y}} = x\bar{y} + xy$ or $(x \oplus y)' = (x'y + xy')' = x'y' + xy$
- Please note that the + in $x + y$ is not ordinary addition, it stands for OR, so that, for example, $1 + 1 = 1$ here. In fact, we have the following rules:
 $0' = 1$ and $1' = 0$
 $0.0 = 0.1 = 1.0 = 0$ and $1.1 = 1$
 $0 + 0 = 0$ and $1 + 0 = 0 + 1 = 1 + 1 = 1$
- Expressions such as $x + y \oplus z'$, $x + (y.x')$ etc., that involve $.$, $+$, $'$, are called Boolean expressions.
- PLEASE NOTE $\overline{XY}$ IS NOT EQUAL TO $\bar{X}\bar{Y}$

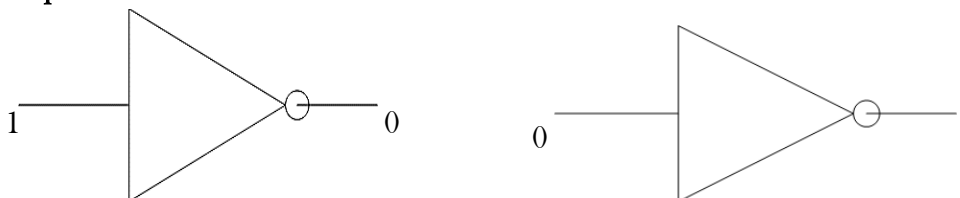
4.3 LOGIC GATES AND CIRCUITS

- Binary electrical signals only have two values, generally taken to be 0 and 1 (or maybe *false* and *true*, depending on how the signals are interpreted).
- The two signal values are implemented using two separate voltage levels. For example, a 5 Volt signal might represent a 1 and a 0 Volt signal might represent a 0.

- The two voltage levels are not standard – they might be 12 Volts and 5 Volts in some systems – but there are always two levels.
- Usually, the higher voltage level represents the 1 and the lower voltage represents the 0. *This is called positive logic.*
- It is possible to use the high voltage level to represent the 0 and the lower voltage level to represent the 1. *This negative logic.*
- *We will assume in this course that positive logic is used.*
- The electronic circuits in a computer are made up of tiny components called transistors.
- A transistor is an electronic device that can block or control the flow of electric current. *These transistors can be combined in various ways to make complex electronic circuits that can do useful things.*
- Electronic circuits made up of transistors can be constructed in such a way that they will implement the logical operators described previously. These electronic circuits are called *logic gates*.
- When working with these gates, we represent them using symbols, and *we ignore the internal electronic components of the gates.*
- There are different kinds of logical gates which includes: basic gates, universal gates and derived gates.
 - The basic logical operators are: NOT, AND, OR.
 - The universal gates are NAND, NOR.
 - The derived gates: XOR, XNOR.
- The following sketch shows the symbols used for the AND gate (left), the OR gate (centre) and the NOT gate (right).

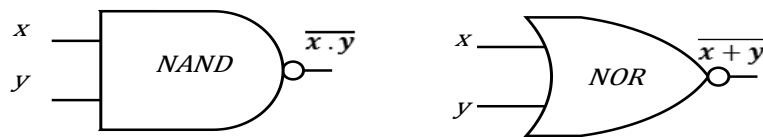


- Let us look at the NOT gate on the right. There is one input wire on the left, and one output wire coming out on the right. *If the input has a high voltage (representing a 1) then the output will have a low voltage (representing a 0).* The internal circuitry of the gate guaranteed this. *Similarly, if the input is low, the output will be high.* That can be represented as follows:



- The AND gate has two inputs, each of which can be either 0 or 1. The output will be either 0 or 1, depending on the inputs.

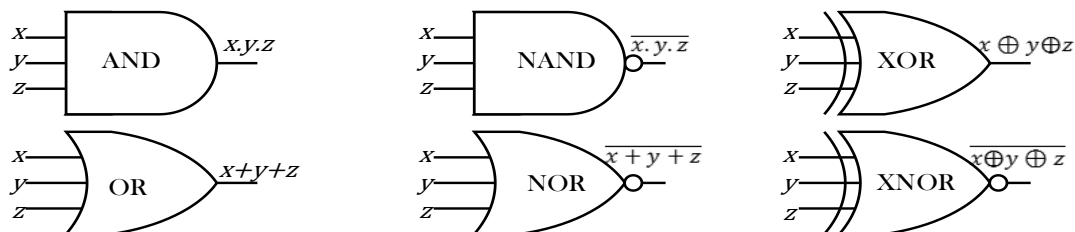
- The following diagram shows the NAND gate (left) and the NOR gate (right), they both take two inputs.



- The following diagram shows the XOR gate (left) and the XNOR gate (right), they both take two inputs.



- It's possible for gates to have more than 2 inputs. For example, the following diagram shows the gates with 3 inputs:

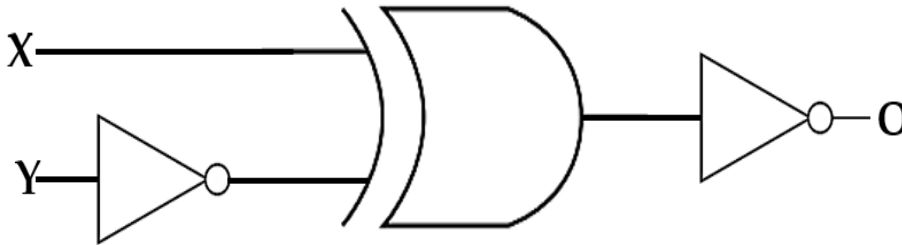


- The diagram below shows the summary of all the gates with their truth tables and symbols.

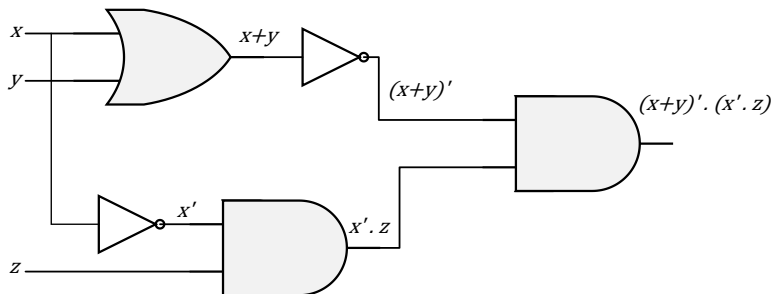
Expression	Symbol	Venn diagram	Boolean algebra	Values		
AND			$A \cdot B$	A	B	Output
				0	0	0
				0	1	0
				1	0	0
				1	1	1
OR			$A + B$	A	B	Output
				0	0	0
				0	1	1
				1	0	1
				1	1	1
XOR			$A \oplus B$	A	B	Output
				0	0	0
				0	1	1
				1	0	1
				1	1	0
NOT			$\bar{A}$	A		Output
				0		1
				1		0
NAND			$\overline{A \cdot B}$	A	B	Output
				0	0	1
				0	1	1
				1	0	1
				1	1	0
NOR			$\overline{A + B}$	A	B	Output
				0	0	1
				0	1	0
				1	0	0
				1	1	0
XNOR			$\overline{A \oplus B}$	A	B	Output
				0	0	1
				0	1	0
				1	0	0
				1	1	1

4.3.1 Logic Circuits Design

- Gates can be connected together to obtain more circuits. For example, *consider the following, made up of two NOT gates and an XOR gate:*

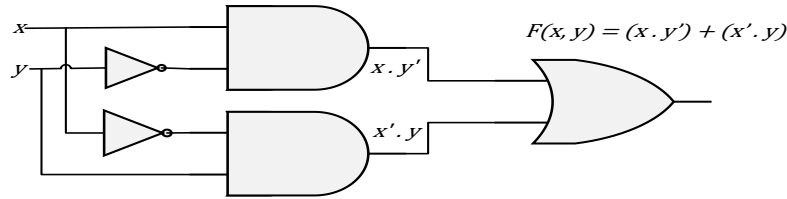


- The original inputs are labelled X and Y. The inputs to the XOR gate are actually X and Y' , since Y goes through a NOT gate first.
 - The output from the XOR gate is then $(X \text{ XOR } Y')$. This output finally goes through a NOT gate, so the entire circuit produces $(X \text{ XOR } Y')'$.
 - What will the output of the circuit be if $X = 1$ and $Y = 1$?
 - The two inputs to the XOR gate are then 1 and $1'$ i.e. 1 and 0.
 - Since $1 \text{ XOR } 0 = 1$, the output from the XOR gate is 1.
 - This passes through a NOT gate, so the final output is 0.
- Here is another example, this time with three inputs X, Y and Z:

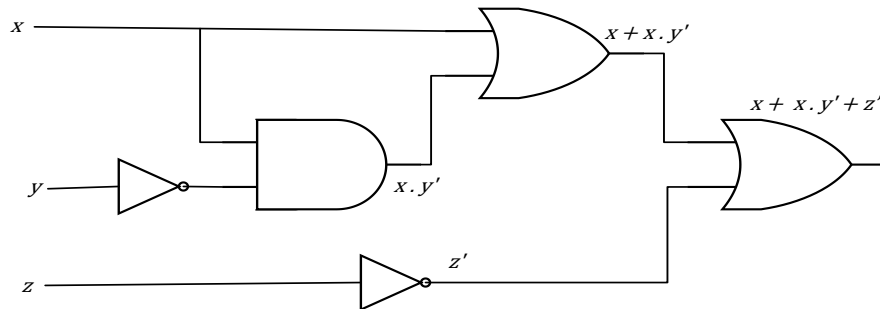


- x and y are the two inputs to the upper OR gate, so its output is $(x + y)$. The two inputs to the lower AND gate are x' (because X feeds through a NOT gate) and z , so the output of the AND gate is $x'.z$
 - Taking into account the central NOT gate, the inputs to the OR gate on the right are $(x + y)'$ AND $x'.z$, so the final output is $(x + y)'.(x'.z)$.
 - Convince yourself that if $x = 0$, $y = 0$ and $z = 1$, then the final output of the circuits is 1.
- We can also go the other way round. Given some logical expression, involving $'$, $.$, $+$ and XOR, we can build a circuit to produce it.
- Consider the expression $(X.Y') + (Y.X')$. We do the brackets first, using two AND gates.
 - The first AND gate has inputs X, Y' and the second AND gate has inputs Y, X' .

- The outputs from these two AND gates then act as inputs to an OR gate, which produces the final result.
- Observe the following output figure:



- Note on the left how the single input for X splits so that the same X is fed into the upper AND gate and also into the NOT gate that is attached to the lower AND gate.
 - If you check the output of the above circuit for various values of the inputs, you will get the following table:
- | x | y | y' | $x \cdot y'$ | x' | $x' \cdot y$ | $F(x, y)$ |
|-----|-----|------|--------------|------|--------------|-----------|
| 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 1 | 0 | 0 | 1 | 1 | 1 |
| 1 | 0 | 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 0 | 0 | 0 | 0 | 0 |
- From this table, you can see that our circuit does exactly the same thing as an XOR gate. So, one can build an XOR gate using two NOT gates, two AND gates and an OR gate.
 - Here is another example, this time using three inputs x, y and z . We will build a circuit that produces $x + x \cdot y' + z'$.
 - The first thing done here is $x \cdot y'$. When we have this, we can get $x + x \cdot y'$ using an OR gate. Then finally $x + x \cdot y' + z'$ is done using a second OR gate. We will also need NOT gates along the way, and this is how it looks:

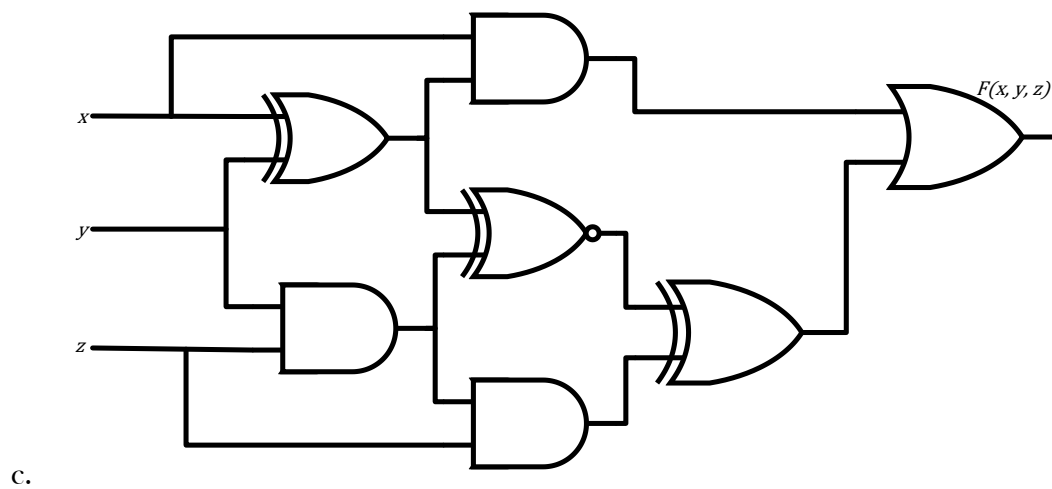
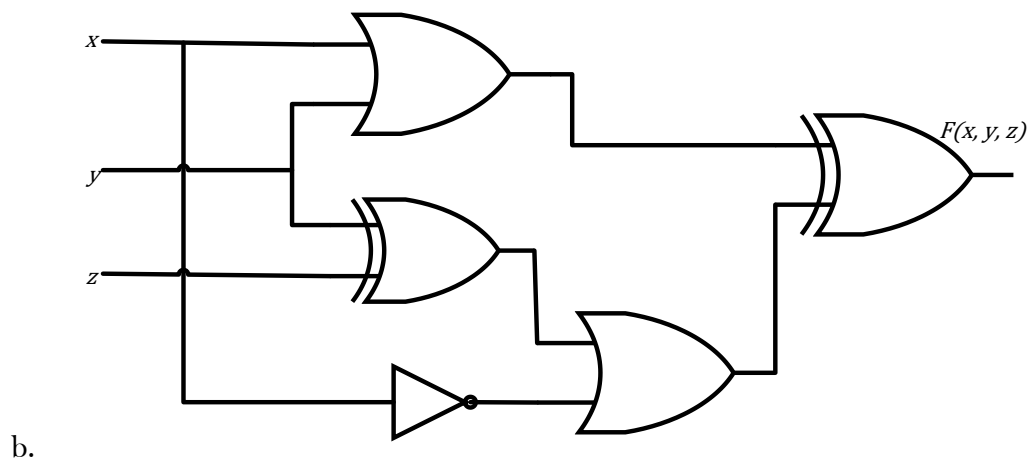
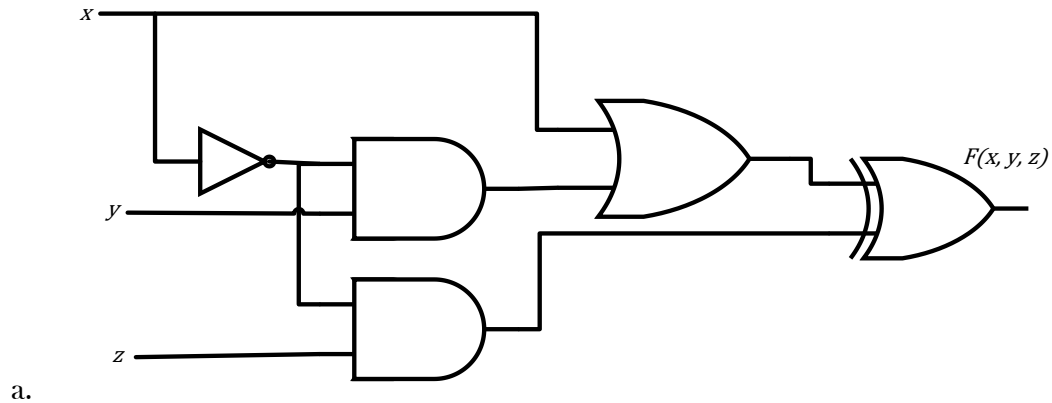


4.3.2 Exercise

- Construct the following operator using only AND, OR and NOT gates:
 - XOR
 - NAND
 - NOR
 - XNOR
- Show that the NOT, OR and AND gates can be constructed using NOR gates.
- Draw the logic circuits for the following expressions:
 - $F(x, y) = (x + y)(x' + y')$
 - $F(x, y) = (x' + y')'$
 - $F(x, y, z) = xy + xz$

- d. $F(x, y, z) = x(y + z)$
- e. $F(x, y, z) = x + y'z$
- f. $F(x, y, z) = xz + (xy + z')$
- g. $F(x, y, z) = x'.y.z' + x'.y.z + x.y.z'$
- h. $F(x, y, z) = xy'z + x'y'z + xyz$
- i. $F(x, y, z) = (xy \oplus (y + z'))' + x'z$

4. Find the truth table of the following circuit:



4.3.3 Truth Tables

- Every Boolean expression has a truth table, which shows the output of the expression for each combination of the inputs.
- The following table for the Boolean expression $X.Y' + X'Y$.
- *The first two columns list all the possible combinations of the inputs X and Y.*
- The required expression is built up bit-by-bit using intermediate columns to lessen the chance of error.

X	Y	X'	Y'	X.Y'	X'Y	X.Y' + X'Y
0	0	1	1	0	0	0
0	1	1	0	0	1	1
1	0	0	1	1	0	1
1	1	0	0	0	0	0

- For an expression such as $(X + Z')'.Y$, which has three inputs, we will need $2^3 = 8$ rows in the table to allow for all combinations of X, Y and Z.
 - *Observe the following output table:*

X	Y	Z	Z'	X + Z'	$(X + Z')'$	$(X + Z')'.Y$
0	0	0	1	1	0	0
0	0	1	0	0	1	0
0	1	0	1	1	0	0
0	1	1	0	0	1	1
1	0	0	1	1	0	0
1	0	1	0	1	0	0
1	1	0	1	1	0	0
1	1	1	0	1	0	0

- Note the pattern of values in the first three columns, *which ensures that all combinations of X, Y, Z are present.*
- Two Boolean expressions are called (logically) equivalent if the following is true: *for each combination of the inputs, the two expressions produce exactly the same outputs.*
- Truth tables can be used to determine whether two expressions are equivalent. *(Another method, to be seen later, is to use Boolean Algebra)*

- **Example:**

- Determine whether the expressions $X + Y.X$ and $(X + Y).X$ are equivalent.

- **Solution:**

- Construct the truth table with columns for $X + Y.X$ and $(X + Y).X$

X	Y	Y.X	X + Y	X + Y.X	$(X + Y).X$
0	0	0	0	0	0
0	1	0	1	0	0
1	0	0	1	1	1
1	1	1	1	1	1

- Since the columns for $X + Y.X$ and $(X + Y).X$ are identical, the two expressions are equivalent.
(In fact, they are both equivalent to the simple expression X, if you can notice).

- **Example:**
 - Determine whether the expressions $(X + Y').Z$ and $(X + Y).Z'$ are equivalent.

- **Solution:**
 - Construct the truth table with columns for $(X + Y').Z$ and $(X + Y).Z'$

X	Y	Z	Y'	Z'	$(X + Y')$	$(X + Y)$	$(X + Y').Z$	$(X + Y).Z'$
0	0	0	1	1	1	0	0	0
0	0	1	1	0	1	0	0	0
0	1	0	0	1	0	1	0	1
0	1	1	0	0	0	1	0	0
1	0	0	1	1	1	1	0	1
1	0	1	1	0	1	1	1	0
1	1	0	0	1	1	1	0	1
1	1	1	0	0	1	1	1	0

- The expressions are not equivalent, as the last two columns are not identical.
- *The expressions give different outputs, for example, when $X = 0$, $Y = 1$, and $Z = 0$.*

4.3.4 Exercise

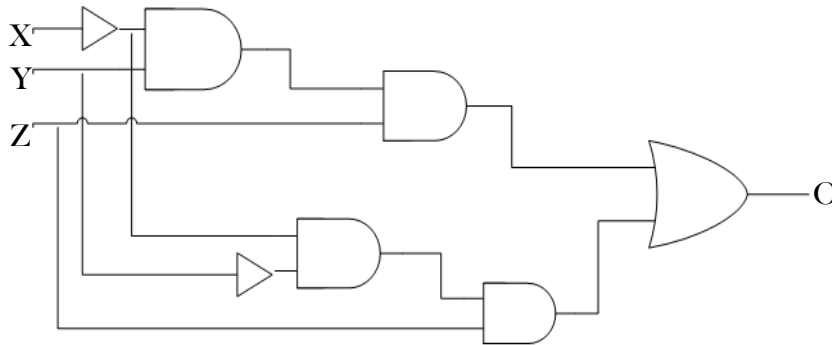
- Construct truth tables for the following expressions:
 - $X.Y.Z + X'.Y'.Z'$
 - $X.Y.Z + X.(Y.Z)' + (X.Y.Z)'$
 - $X.(Y.Z' + Y'.Z)$
 - $(X + Y).(X + Z).(X' + Y')$
 - $X.Y.Z + (\overline{X.Y.Z})$
 - $X.(Y.\bar{Z} + \bar{X}.Y)$
 - $(X + Y).(X + Z).(X' + Z)$
- Show whether each of the following equality is valid or invalid:
 - $\overline{XY} = \bar{X}\bar{Y}$
 - $X \oplus (Y.Z) = (X \oplus Y).(X \oplus Z)$
 - $(X + Y')'.Z = (X'.Y).Z$
- Show that $X = XY + XY'$ using truth tables.

4.3.5 Disjunctive Normal Form

- Suppose we have to design a circuit with three inputs, having the following properties:
 - The output is 1 if either $X = 0, Y = 1, Z = 1$ or $X = 0, Y = 0, Z = 1$. In all other cases the output is 0.
 - How do we design this?
 - The output must be 1 if $X = 0, Y = 1, Z = 1$. Consider the expression $X'.Y.Z$
 - Since this consists of three things “ANDed” together, it can only be 1 if all three are 1, which happens if $X = 0, Y = 1, Z = 1$.
 - Similarly, the expression $X'.Y'.Z$ will produce 1 if X', Y', Z are all 1, i.e. if $X = 0, Y = 0, Z = 1$. If we “OR” these two expressions, we will have an expression for our circuit, which is:

$$X' \cdot Y \cdot Z + X' \cdot Y' \cdot Z$$

- Look at the result logically. This expression gives 1 if either $X' \cdot Y \cdot Z$ gives 1 (i.e. if $X = 0, Y = 1, Z = 1$) or if $X' \cdot Y' \cdot Z$ gives 1 (i.e. if $X = 0, Y = 0, Z = 1$), which is exactly what was required.
- Here is the actual circuit. *The inputs on the left are X, Y, Z from the top to bottom.*



- For another example, suppose we want a circuit with three inputs, X, Y, Z which outputs 1 when:
 - $X = 1, Y = 0, Z = 1$
 - $X = 0, Y = 0, Z = 0$
 - $X = 1, Y = 1, Z = 0$
 and outputs a 0 in all other cases

- Following the same reasoning as before, the required logical expression is:

$$X \cdot Y' \cdot Z + X' \cdot Y' \cdot Z' + X \cdot Y \cdot Z'$$

- Draw the circuit as an exercise.

- The general form of the expression found in these two examples is:
group + group + group +...
- *where each group contains all the input variables, with some negated, ANDed together.* This general form is called a **DISJUNCTIVE NORMAL FORM**. It has some theoretical importance.
- *It turns out that any Boolean expression can be put into an equivalent disjunctive normal form.*
- Let us see how that is done in one case, by finding an equivalent disjunctive normal form for the expression $Z' + (X + Y') \cdot Z$
- First build the truth table for this expression:

X	Y	Z	Y'	Z'	$X + Y'$	$(X + Y') \cdot Z$	$Z' + (X + Y') \cdot Z$
0	0	0	1	1	1	0	1
0	0	1	1	0	1	1	1
0	1	0	0	1	0	0	1
0	1	1	0	0	0	0	0
1	0	0	1	1	1	0	1
1	0	1	1	0	1	1	1

1	1	0	0	1	1	0	1
1	1	1	0	0	1	1	1

- In the final column, pick out all the 1 values and list the combinations of X, Y, Z that give these 1 values. These are (i) $X = 0, Y = 0, Z = 0$ (ii) $X = 0, Y = 0, Z = 1$ etc - in fact all except $X = 0, Y = 1, Z = 1$.
- Now use these seven combinations to build the disjunctive normal form which is:

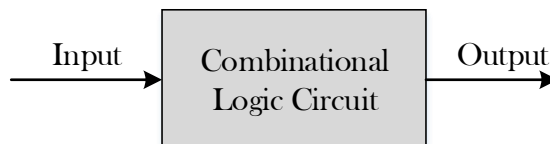
$$X'.Y'.Z' + X'.Y'.Z + X'.Y.Z' + X'.Y.Z + X.Y'.Z + X.Y.Z' + X.Y.Z$$
- If we look at the table for XOR, which only has two inputs:

X	Y	$X \oplus Y$
0	0	0
0	1	1
1	0	1
1	1	0

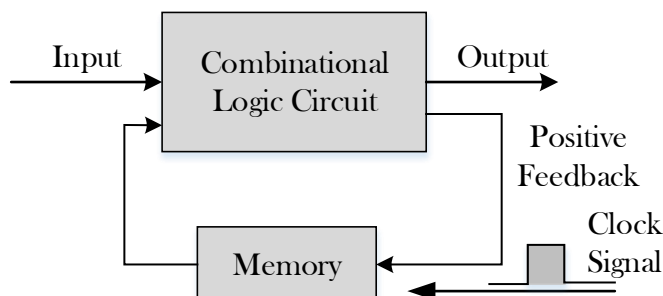
- we see that $X \oplus Y$ is just equal 1 when $X = 0, Y = 1$ or when $X = 1, Y = 0$, so the equivalent disjunctive normal form for $X \text{ XOR } Y$ is $X'.Y + X.Y'$

4.4 BASIC DIGITAL LOGIC CIRCUITS

- Digital logic chips are combined to make useful circuits. The circuits can be categorized as either:
 - Combinational logic circuits
 - Sequential logic circuits
- Combinational and Sequential circuits are the most essential concepts to be understood in digital electronics.
- *Combinational logic is a type of digital logic which is implemented by Boolean circuits, where the output is a pure function of the present input only.* The outputs of combinatorial circuits are computed entirely from their present inputs



- Sequential logic is a type of input whose output depends not only on the present value of its input signals but on the sequence of past inputs. The outputs of sequential circuits are computed using both the present input and their previous output:
 - The idea of a next state
 - A clock input triggers the transition from the current state to next state.



- The following table gives a brief comparison of combinational circuits and sequential circuits

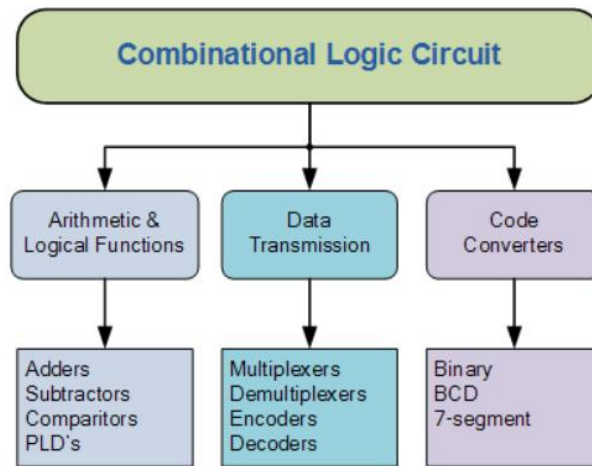
Combinational Logic Circuits	Sequential Logic Circuits
Output is a function of present inputs	Output is a function of clock, present inputs and the previous states of the system
Do not have the ability to store data	Have memory to store the present state that is sent as control input (enable) for the next operation
It does not require feedback. It simply outputs the input according to the logic designed	It involves feedback from output to input that is stored in the memory for the next operation
Used mainly in Arithmetic and Boolean operations	Used for storing data e.g. in the RAM
Logic gates are the elementary building blocks	Flip flops (binary storage device) are the elementary building
Independent of clock and hence does not require triggering to operate	Clocked (Triggered for operation with electronic pulses)
E.g. Half Adder	E.g. counter

- In sequential logic, if memory element is clocked, then the circuit is synchronous, if not, the circuit is asynchronous:
 - Synchronous
 - Properly designed system with no timing problems
 - Asynchronous
 - Timing problems which makes the design of asynchronous sequential circuits difficult
 - Used when speed of operation is important
 - Fast response to the change of input variable
 - No clock distribution

4.4.1 Combinational Circuits

- Combinational Logic Circuits are made up from basic logic NAND, NOR or NOT gates that are “combined” or connected together to produce more complicated switching circuits. These logic gates are the building blocks of combinational logic circuits.
- An example of a combinational circuit is a decoder, which converts the binary code data present at its input into a number of different output lines, one at a time producing an equivalent decimal code at its output.
- Combinational logic circuits can be very simple or very complicated and any combinational circuit can be implemented with only NAND and NOR gates as these are classed as “universal” gates.
- The three main ways of specifying the function of a combinational logic circuit are:
 - **Boolean Algebra** – This forms the algebraic expression showing the operation of the logic circuit for each input variable either True or False that results in a logic “1” output.
 - **Truth Table** – A truth table defines the function of a logic gate by providing a concise list that shows all the output states in tabular form for each possible combination of input variable that the gate could encounter.

- **Logic Diagram** - This is a graphical representation of a logic circuit that shows the wiring and connections of each individual logic gate, represented by a specific graphical symbol, that implements the logic circuit.
- The following diagram shows a classification of combinational circuits.



- As combinational logic circuits are made up from individual logic gates only, they can also be considered as “decision making circuits” and combinational logic is about combining logic gates together to process two or more signals in order to produce at least one output signal according to the logical function of each logic gate.
- Because of time, we are only going to look at a few arithmetic circuits.

4.4.1.1 The Half Adder

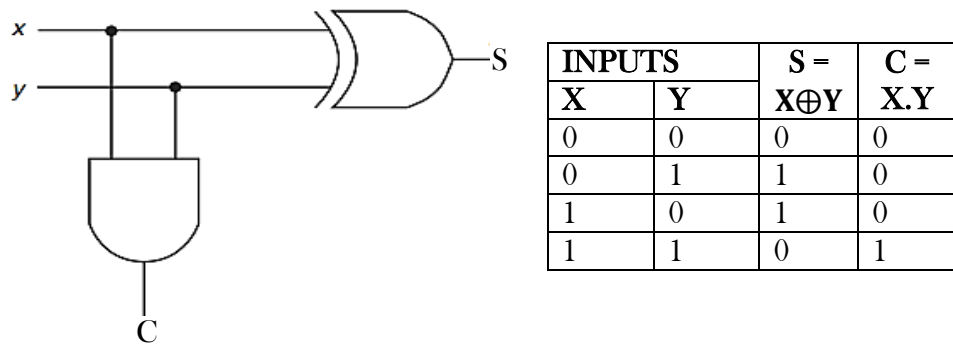
- In this section, we will provide a circuit that adds 2 bits, and outputs the correct sum and carry.
- If we add 2 bits X, Y, each of which can be 0 or 1, we will get a sum S and a carry C e. g.

$$\begin{array}{r}
 1 \quad 1 \quad X \\
 \underline{0} \quad \underline{1} \quad \underline{Y} \\
 01 \quad 10 \quad CS
 \end{array}$$

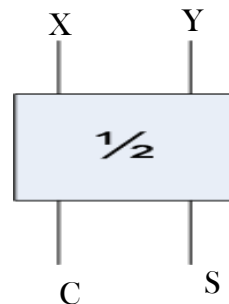
- It is simple to check that the following table gives the correct sum and carry in all cases:

X	Y	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

- It is obvious from the table that $S = X \text{ XOR } Y$ and $C = X.Y$ so we have the following circuit, with two inputs X, Y and two outputs S, C.
 - Observe the follow following circuit together with its truth table:



- The circuit is known as a half adder. *It is often represented as follows, with the understanding that inside the box labelled $\frac{1}{2}$ is the circuitry shown above:*

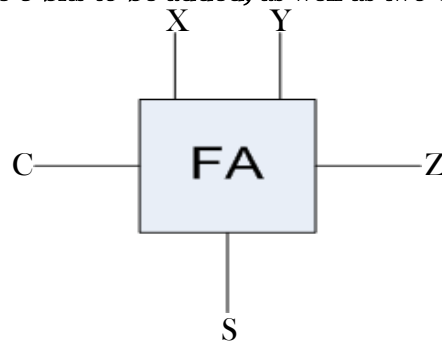


If $X = 1$ and $Y = 1$ as inputs, then the outputs will be $S = 0$ and $C = 1$.

- Draw the circuit of the Half Adder using the NAND gates only.

4.4.1.2 The Full Adder

- The half adder cannot do very much. *To get something very useful, we need a circuit that will add 3 bits and produce the correct sum and carry.*
- In abstract terms, the object we will design is shown below. *It has three inputs, X, Y, Z, which are the 3 bits to be added, as well as two outputs, the sum S and the carry C.*



- We need to work out what circuitry should go inside the box labelled FA. *The process of adding 3 bits X, Y, Z can be carried out in three steps:*
 - Add X to Y to get a sum S1 and a carry C1
 - Add Z to S1 to get a sum S and a new carry C2
 - Add the two carries to get a final carry C
- Here are two examples to illustrate:
 - First let us add 1 plus 1 plus 0 ($X = Y = 1$ and $Z = 0$):

1	X
1	Y
<u> </u>	

$$\begin{array}{rcl}
 (1) & 0 & S1 = 0 \text{ and } C1 = 1 \\
 & \underline{0} & Z \\
 (0) & 0 & S = 0 \text{ and } C2 = 0
 \end{array}$$

Final answer is 10 after adding the two carries to get C.

- Next let us add 1 plus 1 plus 1 ($X = Y = Z = 1$):

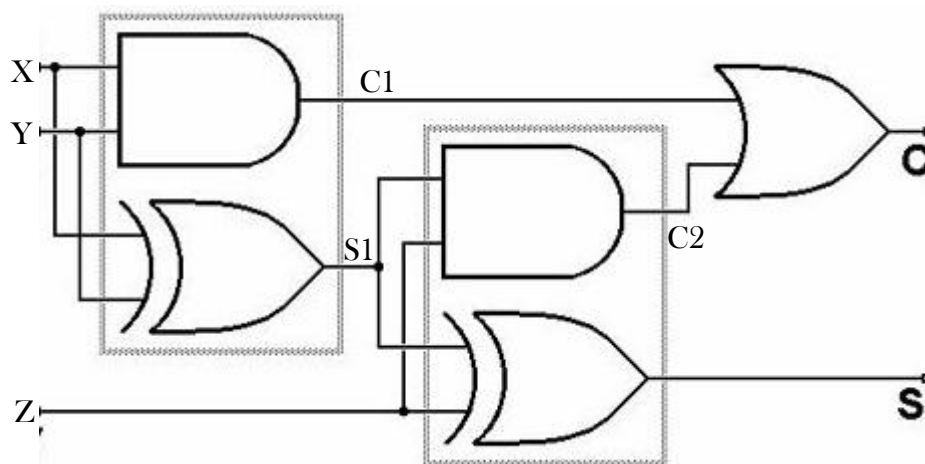
$$\begin{array}{rcl}
 & 1 & X \\
 & \underline{1} & Y \\
 (1) & 0 & S1 = 0 \text{ and } C1 = 1 \\
 & \underline{1} & Z \\
 (0) & 1 & S = 1 \text{ and } C2 = 0
 \end{array}$$

Final answer is 11 after adding the two carries.

- We can add X to Y using a half adder which will produce outputs $S1$ and $C1$. We can then feed $S1$ and Z through a second half adder to produce the outputs S and $C2$.
- We could then add the two carries $C1$ and $C2$ using a third half adder, but there is a simpler way to combine the two carries.
- A few examples will show that it is not possible for $C1$ and $C2$ to be equal to 1. The only possibilities are contained in the following table:

$C1$	$C2$	C
0	0	0
0	1	1
1	0	1

- From this table we see that $C = C1 \text{ OR } C2$ in all the cases that can actually occur.
- We can therefore obtain C from $C1$ and $C2$ using an OR gate, which is cheaper and less complicated than a half adder.
- Here is the full circuit:



- The two half adders contain the circuitry (XOR and an AND gate) as described in the previous section.
- This circuit (with three inputs X, Y, Z and two outputs S, C) is known as the full adder.
- The truth table for the full adder is as follows:

INPUTS	$S1 =$	$S =$	$C1 =$	$C2 =$	$C =$
--------	--------	-------	--------	--------	-------

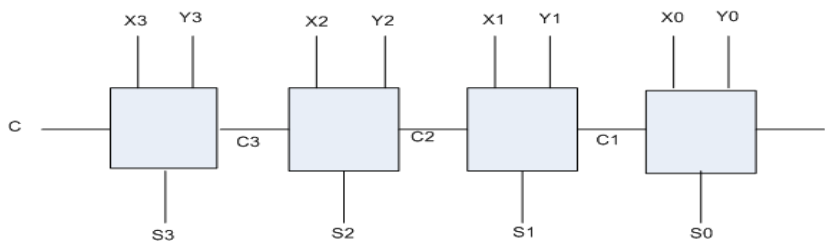
X	Y	Z	$X \oplus Y$	$S1 \oplus Z$	$X \cdot Y$	$S1 \cdot Z$	$C1 + C2$
0	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0
0	1	0	1	1	0	0	0
0	1	1	1	0	0	1	1
1	0	0	1	1	0	0	0
1	0	1	1	0	0	1	1
1	1	0	0	0	1	0	1
1	1	1	0	1	1	1	1

- Draw the circuit of the full adder using only the NAND gates.

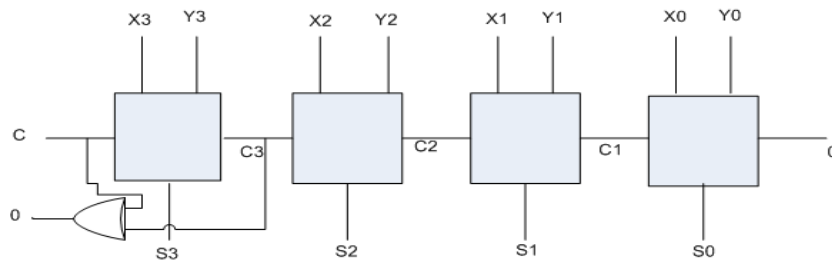
4.4.1.3 A Circuit to Add Two 4-Bit Binary Numbers

- As an example, look at

$$\begin{array}{r} 1011 \\ 1001 \\ \hline (1) 0100 \end{array}$$
- *We want a circuit that will do such a 4-bit addition, producing the correct 4-bit sum and the correct carry.*
- To add these two 4-bit numbers, we do one column at a time, starting from the right.
- *For each column, we add three things: the two numbers in the column, and the “incoming” carry from the previous column. The result is a sum from that column, as well as an “outgoing” carry that will be added into the next column. All this can be accomplished with a full adder. Note that for the first column (the one on the right), the incoming carry is zero.*
- Here is the circuit which adds the two 4-bit numbers $X_3 X_2 X_1 X_0$ and $Y_3 Y_2 Y_1 Y_0$ to get a sum $S_3 S_2 S_1 S_0$ and a final carry C :



- Each box represents a full adder. The box second from the right takes $X1$, $Y1$ and $C1$ as inputs, and produces $S1$ and $C2$ as outputs.
 - The outgoing carry $C2$ will feed into the next full adder as an incoming carry. Note the initial incoming carry of zero to the rightmost full adder.
 - Obviously, if we want to add two 8-bit binary numbers, we need to use eight full adders instead of four.
- Our 4-bit adder clearly shows whether there is a carry – the output C is 1 if the addition produces a final carry. **What about overflow? Overflow occurs if $C3$ and C are different so we can detect overflow by adding an XOR gate as shown below. Output 0 will be 1 if $C3$ and C are different i.e. if overflow occurs.**

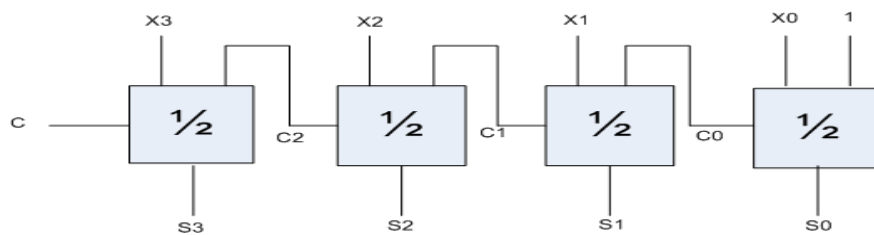


4.4.1.4 A circuit that adds one to a 4-bit binary number

- We can certainly use the 4-bit binary adder described previously above to add 1 to a 4-bit number, by using the 4-bit adder to add $X_3X_2X_1X_0$ to 0001.
- Consider the addition:

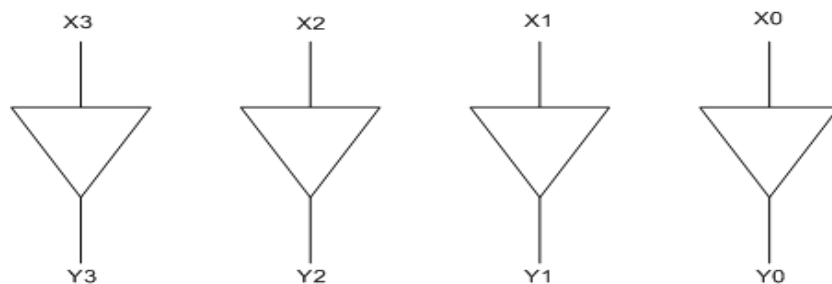
$$\begin{array}{r}
 X_3 \ X_2 \ X_1 \ X_0 \\
 0 \ 0 \ 0 \ 1 \\
 \hline
 C \ S_3 \ S_2 \ S_1 \ S_0
 \end{array}$$

- In the rightmost column, we add 1 to X_0 to get a sum S_0 and a carry C_0 .
- In the next column, we add C_0 to X_1 to get the sum S_1 and a carry C_1 .
- Then we add C_1 to X_2 to get the sum S_2 and carry C_2 .
- Finally, we add C_2 to X_3 to get a sum and carry, and this can be done with a simple half adder.
- Here is the circuit:



4.4.1.5 A circuit that finds the ones complement of a 4-bit binary number

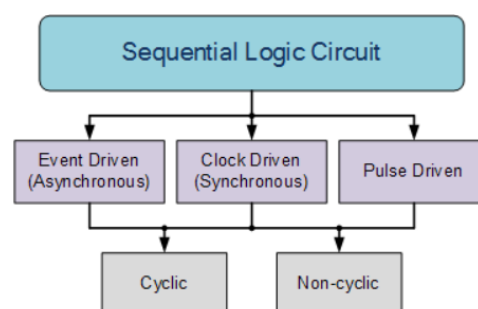
- This is very simple. All we need to do is to reverse the inputs X_0 , X_1 , X_2 , and X_3 using NOT gates:



4.4.2 Sequential Logic Elements

- Sequential logic can perform as many different functions as combinational logic. The sequential circuit makes use of combinational circuits.
- Combinational circuits implement the essential functions of a digital computer. However, except for the special case of ROM, they provide no memory or state information, elements also essential to the operation of a digital computer.

- For the latter purposes, a more complex form of digital logic circuit is used: the sequential circuit.
- The current output of a sequential circuit depends not only on the current input, but also on the past history of inputs.
- Another and generally more useful way to view it is that the current output of a sequential circuit depends on the current input and the current state of that circuit.
- Sequential logic circuits are able to take into account their previous input state as well as those actually present, a sort of “before” and “after” effect is involved with sequential circuits.
- In other words, the output state of a “sequential logic circuit” is a function of the following three states, the “**present input**”, the “**past input**” and/or the “**past output**”.
- *Sequential Logic circuits* remember these conditions and stay fixed in their current state until the next clock signal changes one of the states, giving sequential logic circuits “Memory”.
- Sequential logic circuits are generally termed as two state or Bistable devices which can have their output or outputs set in one of two basic states, a logic level “1” or a logic level “0” and will remain “latched” indefinitely in this current state or condition until some other input trigger pulse or signal is applied which will cause the bistable to change its state once again.
- The word “Sequential” means that things happen in a “sequence”, one after another and in Sequential Logic circuits. The actual clock signal determines when things will happen next.
- Simple sequential logic circuits can be constructed from standard Bistable circuits such as: Flip-flops, Latches and Counters and which themselves can be made by simply connecting together universal NAND Gates and/or NOR Gates in a particular combinational way to produce the required sequential circuit.
- As standard logic gates are the building blocks of combinational circuits, bistable latches and flip-flops are the basic building blocks of sequential logic circuits. Sequential logic circuits can be constructed to produce either simple edge-triggered flip-flops or more complex sequential circuits such as storage registers, shift registers, memory devices or counters. Either way sequential logic circuits can be divided into the following three main categories:
 - **Event Driven** – asynchronous circuits that change state immediately when enabled.
 - **Clock Driven** – synchronous circuits that are synchronized to a specific clock signal.
 - **Pulse Driven** – which is a combination of the two that responds to triggering pulses.



- Examples of sequential circuits includes:
 - **Latch:** a 1-bit memory element
 - **Register:** m latches in a row and is able to store an m-bit word
 - **Shift register:** special-purpose register that moves the bits of the word left or right
 - **Counter:** special-purpose register that when triggered (i.e. clocked) increases its contents by 1
 - **State machines:** moves from one state to another each time it is triggered
- **YOU ARE GOING TO LOOK AT A FEW SEQUENTIAL CIRCUITS AS AN ASSIGNMENT.**

4.5 BOOLEAN ALGEBRA

- Expressions that involve **AND, OR, NOT, XOR** are called **Boolean expressions**.
- If a Boolean expression is very complicated, **requiring a lot of gates**, we may want to try and simplify it, so that less gates are needed.
- This may be done using **Boolean Algebra**.
- Boolean algebra is used to describe the circuits that can be built by combining gates.
- It captures the essence of the logical operations.
- Boolean functions are represented by truth tables.
- There are a number of **basic rules in Boolean Algebra**. Some of these rules are the **same as in ordinary algebra**, while others are new.
- The following table provides the rules. In the table, x , y and z can stand for any Boolean expression:

Rule Number	Rule	Name
Rule 1	$x + y = y + x$ $x \cdot y = y \cdot x$	Commutative Laws
Rule 2	$(x + y) + z = x + (y + z)$ $(x \cdot y) \cdot z = x \cdot (y \cdot z)$	Associative Laws
Rule 3	$x \cdot (y + z) = x \cdot y + x \cdot z$ $x + y \cdot z = (x + y) \cdot (x + z)$	Distributive Laws
Rule 4	$(x + y)' = x' \cdot y'$ $(x \cdot y)' = x' + y'$	de Morgan's Laws
Rule 5	$x + x \cdot y = x$ $x \cdot (x + y) = x$	Absorption Law
Rule 6	$(x')' = x$	Involution Law
Rule 7	$0 + x = x$ $1 \cdot x = x$	Identity Elements
Rule 8	$1 + x = 1$ $0 \cdot x = 0$	Null Elements
Rule 9	$x + x = x$ $x \cdot x = x$	Idempotence
Rule 10	$x + x' = 1$ $x \cdot x' = 0$	Complementary Law

- The 10 rules in the previous table can be proved using truth tables, or by inspection.

- For example, to prove Rule 9, look at the cases for x : if $x = 0$, then $x + x = 0 + 0 = 0 = x$ and if $x = 1$, the $x + x = 1 + 1 = 1 = x$.
- A truth table to prove the second of de Morgan's Laws is as follows:

x	y	x'	y'	$x.y$	$(x.y)'$	$x' + y'$
0	0	1	1	0	1	1
0	1	1	0	0	1	1
1	0	0	1	0	1	1
1	1	0	0	1	0	0

4.5.1 Comments on the rules

- The first three rules are very like ordinary algebra, *except for the second part of Rule 3, which is strange, and should be noted.*
- The first rule allows us to change the order of things e. g. $x + y + z$ is the same as $x + z + y$, and $x.y.z$ is the same as $x.z.y$.
- The second rule means that we can leave out brackets in the expressions like $x + y + z$, *since it does not matter whether it is treated as $(x + y) + z + c$ or $x + (y + z)$.*

- The third rule gives us the law for removing brackets. They are variations on this law e. g.:

$$\begin{aligned}
 (x + y).z &= z.(x + y) && \text{[Rule 1]} \\
 &= z.x + z.y && \text{[Rule 3]} \\
 &= x.z + y.z && \text{[Rule 1]}
 \end{aligned}$$

which gives us a new rule, but it is really just a combination of Rules 1 and 3.

- Also as above $(w + x).(y + z) = (y + z).(w + x)$ [Rule 1]
 $= (y + z).w + (y + z).x$ [Rule 3]
 $= w.(y + z) + x.(y + z)$ [Rule 1]
 $= w.y + w.z + x.y + x.z$ [Rule 3]

$$\begin{aligned}
 \text{and } w + x.y.z &= (w + x).(w + y.z) && \text{[Rule 3]} \\
 &= (w + x).(w + y).(w + z) && \text{[Rule 3]}
 \end{aligned}$$

- All of these rules for removing brackets can be regarded as versions of Rule 3. Rule 4, de Morgan's Laws, is used very frequently, as we will see.
- Some rules can actually be derived from others. For example, in Rule 5,

$$\begin{aligned}
 x + x.y &= (x + x).(x + y) && \text{[Rule 3]} \\
 &= x.(x + y) && \text{[Rule 9]}
 \end{aligned}$$

which gives us part of Rule 5

- In any of the rules, x , y and z can be replaced by any Boolean expressions.
- For example:

$$x.y + (x.y)' = 1 \quad \text{[Rule 9]}$$

$$(x.y + z)' = (x.y)' . z' \quad [\text{Rule 4}]$$

- More Examples:

$$(x + y')' = x' . (y')' \quad [\text{Rule 4}]$$

$$= x' . y \quad [\text{Rule 6}]$$

$$(x.y' + z'.w')' = (x.y')' . (z'.w')' \quad [\text{Rule 4}]$$

$$= (x' + (y')') . ((z')' + (w')') \quad [\text{Rule 4}]$$

$$= (x' + y) . (z + w) \quad [\text{Rule 6}]$$

$$= x'.z + y.z + x'.w + y.w \quad [\text{Rule 3}]$$

(In fact, line 3 here is simpler than line 4 – line 3 needs four gates, line 4 needs nine. Removing the brackets is not always a good idea.)

- Another Example:

$$z.x.w' + z.x.w + y.z + y = z.x.(w' + w) + y.z + y \quad [\text{Rule 3}]$$

$$= z.x.1 + y.z + y \quad [\text{Rule 10}]$$

$$= z.x + y.z + y \quad [\text{Rule 1 and 7}]$$

$$= z.x + y + y.z \quad [\text{Rule 1}]$$

$$= z.x + y \quad [\text{Rule 5}]$$

(In a test, we would not mind if you skipped very obvious steps such as going directly from $y.z + y$ to y without bothering to change the order to $y + y.z$)

- When dealing with XOR, use the rule that $x \oplus y = x.y' + y.x'$
- The following example contains an unexpected trick:

$$x \oplus y + x.y = x.y' + y.x' + x.y \quad [\text{Rule for XOR}]$$

$$= x.(y'.y) + y.x' \quad [\text{Rules 3 and 1}]$$

$$= x.1 + y.x' \quad [\text{Rule 10}]$$

$$= x + y.x' \quad [\text{Rules 7 and 1}]$$

$$= x + x.y + y.x' \quad [\text{Rule 5}]$$

$$= x + y.(x + x') \quad [\text{Rules 3 and 1}]$$

$$= x + y.1 \quad [\text{Rule 10}]$$

$$= x + y \quad [\text{Rule 7 and 1}]$$

- Here is another example with the trick of “adding something in”:

$$w.x + w'.y + x.y.z = w.x + w'.y + 1.x.y.z \quad [\text{Rule 7}]$$

$$= w.x + w'.y + (w + w').x.y.z \quad [\text{Rule 10}]$$

$$= w.x + w'.y + w.x.y.z + w'.x.y.z \quad [\text{Rule 3}]$$

$$= w.x.1 + w.x.y.d + w'.y.1 + w'.y.x.z \quad [\text{Rules 7 \& 1}]$$

$$= w.x.(1 + y.z) + w'.y.(1 + x.z) \quad [\text{Rule 3}]$$

$$= w.x.1 + w'.y.1 \quad [\text{Rule 8}]$$

$$= w.x + w'.y \quad [\text{Rule 7}]$$

- Given a Boolean function, its implementation is done in the following way:
 1. Write down the truth table for the function.
 2. Provide inverters to generate the complement of each input.
 3. Draw an AND gate for each term with a 1 in the result column (from the truth table).

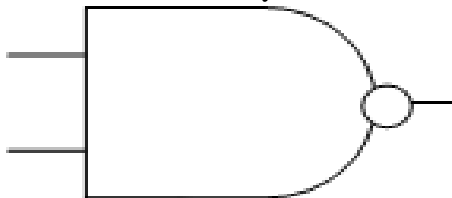
4. Wire the AND gates to the appropriate inputs.
5. Feed the output of all the AND gates into an OR gate

4.5.2 Exercise

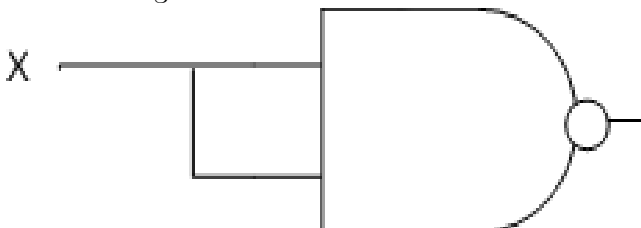
1. Simplify the following functions using identities. In each step, state the identity used:
 - a) $F(x, y) = x + xy$
 - b) $F(x, y) = xy + xy'$
 - c) $F(x, y, z) = \bar{x}yz + \bar{x}y\bar{z} + xz$
 - d) $F(x, y, z) = xy + \bar{x}z + yz$
 - e) $F(x, y, z) = x'y + xyz' + xyz$
 - f) $F(x, y, z) = (x + y)'(x' + y)'$
 - g) $F(x, y, z) = x'yz + xz$
 - h) $F(x, y, z) = (\bar{x} + \bar{y})(\bar{x} + \bar{y})$
 - i) $F(x, y, z) = \bar{x}\bar{x}y$
 - j) $F(x, y, z) = \bar{x}yz + xz$
2. Prove that the following equalities are true using identities:
 - a) $(x + y)(x + \bar{y})(\bar{x} + z) = xz$
 - b) $x = xy + x\bar{y}$
3. Prove whether the following equalities are true or false:
 - a) $yz + xyz' + x'y'z = xy + x'z$
 - b) $x(x' + y) = xy$
 - c) $xy + x'z + yz = xy + x'z$
4. Simplify and draw the logic diagram of the following function:
 - a) $F(x, y, z) = xy'z + x'y'z + xyz$

4.5.3 The NAND Gate

- Remember $X \text{ NAND } Y$ means the same as $\text{NOT } (X \text{ AND } Y)$ i.e. $(X \cdot Y)'$. It is clear that $X \text{ NAND } Y = 0$ when $X = Y = 1$ and $X \text{ NAND } Y = 1$ otherwise.
- Also recall that the symbol for the NAND gate is:

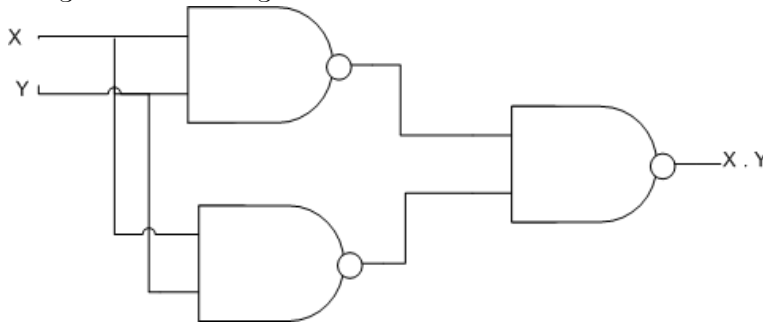


- All the other gates can be constructed using NAND gates. Before we demonstrate this, note that, to use a NAND gate, you need an expression of the form $(X \cdot Y)'$.
- If you want to show that some Boolean expression can be expressed using NAND gates, you need to get the expression into the form $(X \cdot Y)'$, and X, Y must themselves be of that form.
- First $X' = (X \cdot X)'$ so the NOT gate can be built using a single NAND gate as shown in the following slide:

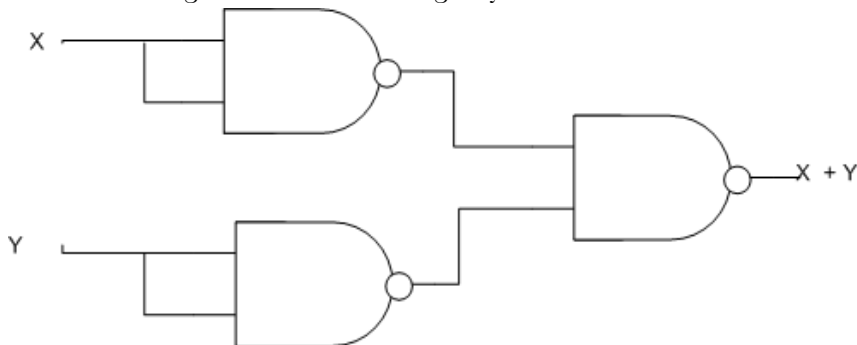


- The input is X and the output is $(X \cdot X)' = X'$.

- Next $X \cdot Y = ((X \cdot Y)')' = ((X \cdot Y)' \cdot (X \cdot Y)')'$, and this allows us to construct an **AND** gate using three **NAND** gates as follows:



- Finally, $X + Y = (X' \cdot Y')' = ((X \cdot X)' \cdot (Y \cdot Y)')'$ and we can construct an **OR** gate from three **NAND** gates in the following way:



- Example:**
 - Show that $X \cdot Y' + X'$ can be constructed using NAND gates.

$$X \cdot Y' + X' = ((X \cdot Y')' \cdot X')' \quad \text{[de Morgan's Laws]}$$

$$= ((X \cdot (Y \cdot Y)')' \cdot (X \cdot X)')'$$
 which can be built using four NAND gates.

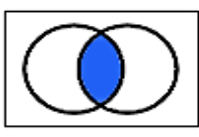
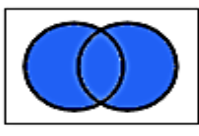
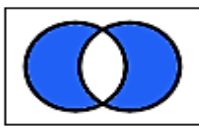
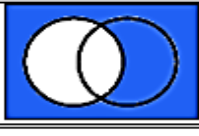
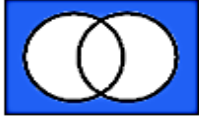
4.5.4 Exercise

- Construct the following using NAND gates:
 - $X + Y'$
 - $X \cdot Y \cdot Z$
 - $X \oplus Y = X \cdot Y' + Y \cdot X'$
 - $X \cdot Y' + X'$
- All the other gates can be constructed using the NOR gates as well. Construct all the circuits in (1) using NOR gates.

RULES FOR BOOLEAN ALGEBRA

Rule Number	Rule	Name
Rule 1	$x + y = y + x$ $x \cdot y = y \cdot x$	Commutative Laws
Rule 2	$(x + y) + z = x + (y + z)$ $(x \cdot y) \cdot z = x \cdot (y \cdot z)$	Associative Laws
Rule 3	$x \cdot (y + z) = x \cdot y + x \cdot z$ $x + y \cdot z = (x + y) \cdot (x + z)$	Distributive Laws
Rule 4	$(x + y)' = x' \cdot y'$ $(x \cdot y)' = x' + y'$	de Morgan's Laws
Rule 5	$x + x \cdot y = x$ $x \cdot (x + y) = x$	Absorption Law
Rule 6	$(x')' = x$	Involution Law
Rule 7	$0 + x = x$ $1 \cdot x = x$	Identity Elements
Rule 8	$1 + x = 1$ $0 \cdot x = 0$	Null Elements
Rule 9	$x + x = x$ $x \cdot x = x$	Idempotence
Rule 10	$x + x' = 1$ $x \cdot x' = 0$	Complementary Law

LOGIC GATES SYMBOLS

Expression	Symbol	Venn diagram	Boolean algebra	Values		
AND			$A \cdot B$	A	B	Output
				0	0	0
				0	1	0
				1	0	0
				1	1	1
OR			$A + B$	A	B	Output
				0	0	0
				0	1	1
				1	0	1
				1	1	1
XOR			$A \oplus B$	A	B	Output
				0	0	0
				0	1	1
				1	0	1
				1	1	0
NOT			$\overline{A}$	A		Output
				0		1
				1		0
NAND			$\overline{A \cdot B}$	A	B	Output
				0	0	1
				0	1	1
				1	0	1
				1	1	0
NOR			$\overline{A + B}$	A	B	Output
				0	0	1
				0	1	0
				1	0	0
				1	1	0
XNOR			$\overline{A \oplus B}$	A	B	Output
				0	0	1
				0	1	0
				1	0	0
				1	1	1